

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск набора подстрок в строке
Вариант 3

Студент гр. 4345

Кравцов С. А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритм Ахо-Корасик

Постановка задачи

Часть 1. Разработайте программу, решающую задачу точного поиска набора образцов.

Ввод: Первая строка содержит текст ($T, 1 \leq |T| \leq 100000$). Вторая - число $n (1 \leq n \leq 3000)$, каждая следующая из n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\} 1 \leq |p_i| \leq 75$. Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Вывод: Все вхождения образцов из P в T . Каждое вхождение образца в текст представить в виде двух чисел - i и p . Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1). Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

Ввод

NTAG

3

TAGT

TAG

T

Вывод

2 2

2 3

Часть 2. Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с *джокером*. В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблону образцу P необходимо найти все вхождения P в текст T . Например, образец $ab??c?a$ с джокером $??$ встречается дважды в тексте $xabvccbababcsaxxabvccbababcsax$.

Символ джокер не входит в алфавит, символы которого используются в T. Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида ??? недопустимы. Все строки содержат символы из алфавита {A,C,G,T,N}

Вход:

Текст (T, $1 \leq |T| \leq 100000$)

Шаблон (P, $1 \leq |P| \leq 40$)

Символ джокера

Выход: Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер). Номера должны выводиться в порядке возрастания.

Ввод

ACTANCA

A\$ \$A\$

\$

Вывод

1

Вариант 3. Вычислить длину самой длинной цепочки из суффиксных ссылок и самой длинной цепочки из конечных ссылок в автомате.

Выполнение работы

При наивном поиске нескольких подстрок в тексте можно было бы для каждого шаблона независимо запускать алгоритм наподобие простого сопоставления, что в худшем случае даёт сложность порядка $O(n \cdot \sum m_i)$. Это неэффективно, так как одинаковые участки текста анализируются многократно. Алгоритм Ахо-Корасик решает эту проблему, объединяя все шаблоны в одну структуру — бор (префиксное дерево), а затем дополняя его автоматными переходами и специальными ссылками. Это позволяет обрабатывать текст за линейное время $O(n + \sum m_i)$, не возвращаясь назад по тексту и не дублируя вычисления.

Были реализованы методы построения бора, вычисления суффиксных и выходных ссылок, а также поиска в тексте.

Построение бора (`add_string`) — на этом этапе все шаблоны добавляются в дерево. Процесс начинается с корня и последовательно обрабатывает символы строки. Если переход по символу отсутствует, создаётся новый узел, в противном случае используется уже существующий. Таким образом, общие префиксы строк хранятся единожды. В конце каждой строки узел помечается как терминальный (`is_leaf`), и в нём сохраняется номер шаблона. Это формирует компактное представление всех шаблонов с общей сложностью $O(\sum m_i)$.

Суффиксные ссылки (`get_suff_link`) — аналог префикс-функции в КМП, но для дерева. Для каждой вершины определяется ссылка на наибольший собственный суффикс строки, соответствующей пути к этой вершине, который также является префиксом одного из шаблонов. Если вершина является корнем или её родитель — корень, ссылка ведёт в корень. В противном случае она вычисляется рекурсивно через переходы автомата. Эти ссылки позволяют эффективно “откатываться” при несовпадении, не возвращаясь к началу.

Автоматные переходы (`get_link`) — реализуют функцию перехода автомата. Если из текущего узла есть ребро по символу, используется оно. Иначе, если узел — корень, остаётся в корне. В противном случае переход осуществляется по суффиксной ссылке. Благодаря мемоизации (словарь `go`),

каждый переход вычисляется не более одного раза, что сохраняет линейную сложность.

Выходные (конечные) ссылки (`get_up`) — позволяют находить все шаблоны, которые оканчиваются в текущей позиции. Если суффиксная ссылка указывает на терминальную вершину, то она и есть ближайшее совпадение. Иначе происходит переход по цепочке дальше. Это даёт возможность находить перекрывающиеся вхождения без повторного обхода.

Поиск в тексте (`process_text`) — основной алгоритм проходит по тексту один раз. Указатель `i` движется слева направо, а текущее состояние автомата обновляется через `get_link`. После каждого перехода проверяется текущая вершина и вся цепочка выходных ссылок (`up`), что позволяет обнаружить все шаблоны, заканчивающиеся в данной позиции. Индексы вхождений вычисляются с учётом длины соответствующего шаблона. За счёт использования ссылок ни один символ текста не обрабатывается более константного числа раз, что обеспечивает сложность $O(n)$.

Вывод автомата (`print_automaton`) — выполняется рекурсивный обход бора с отображением структуры, включая суффиксные и выходные ссылки, а также терминальные вершины.

Часть 2 решает задачу поиска одного шаблона с wildcard-символами, что приводит к ряду изменений.

Во-первых, шаблон предварительно разбивается на части без wildcard'ов. В бор добавляются не сами шаблоны, а эти подстроки, причём для каждой сохраняется её позиция (`offset`) в исходном шаблоне.

Во-вторых, меняется обработка совпадений: если раньше найденный терминальный узел сразу давал готовое вхождение, то теперь это лишь совпадение части. Поэтому используется массив `counts`, который накапливает количество совпавших частей для каждой позиции в тексте.

В-третьих, итоговое совпадение определяется только после обработки всего текста: позиция считается корректной, если в ней совпали все части шаблона (`counts[i] ==` число частей).

Таким образом, основное отличие — переход от прямого поиска шаблонов к схеме “разбиение → поиск частей → сборка результата”, что позволяет учитывать wildcard-символы без потери линейной сложности.

Сложность:

Базовый Ахо-Корасик:

По времени: $O(n + \sum m_i)$, n — длина текста, m_i -длины шаблонов

По памяти: $O(\sum m_i)$, m_i -длины шаблонов

Wild Ахо-Корасик:

По времени: $O(n + \sum m_i + k)$, n — длина текста, m_i -длины подшаблонов, k — длина шаблона

По памяти: $O(n + \sum m_i)$, m_i -длины подшаблонов, n -длина массива counts.

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1-2.

Таблица 1 – Тестирование ас

№	Входные данные	Выходные данные
1	ababa	1 1
	2	2 2
	aba	3 1
	ba	4 2
2	aaaa	1 1
	3	1 2
	a	1 3
	aa	2 1
	aaa	2 2
		2 3
		3 1
		3 2
3	ababcd	4 1
	3	1 1
	abc	1 3
	abcd	4 1
	ab	4 2
		4 3

Таблица 2 – Тестирование as_ws

№	Входные данные	Выходные данные
1	abacabadaba a\$a \$	1 3 5 7 9
2	aaaa \$\$\$ \$	1 2 3 4

Выводы

В ходе работы были реализованы алгоритмы строкового поиска на базе автомата Ахо–Корасик с линейными характеристиками сложности $O(n + \sum m_i)$. Первый вариант обеспечивает поиск множества шаблонов в тексте с использованием бора, суффиксных и выходных ссылок, что позволяет находить все вхождения, включая перекрывающиеся, без возврата по тексту. Второй вариант расширяет функциональность алгоритма, добавляя поддержку шаблонов с wildcard-символами за счёт разбиения шаблона на части и последующей агрегации совпадений. Оба подхода оптимизированы по времени и памяти, исключают избыточные сравнения и обеспечивают эффективную обработку строковых данных даже при сложных условиях поиска.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл ас.ру

```
class Node:
    _node_id_counter = 0

    def __init__(self, parent=None, char_to_parent=None):
        self.id = Node._node_id_counter
        Node._node_id_counter += 1
        self.son: dict[str, 'Node'] = {}
        self.go: dict[str, 'Node'] = {}
        self.parent: 'Node' = parent
        self.char_to_parent: str = char_to_parent
        self.suff_link: 'Node' = None
        self.up: 'Node' = None
        self.is_leaf: bool = False
        self.leaf_pattern_numbers: list[int] = []

class AhoCorasick:
    def __init__(self):
        Node._node_id_counter = 0
        self.root = Node()
        self.pattern_lengths = {}

    def add_string(self, s, pattern_number):
        cur = self.root
        for char in s:
            if char not in cur.son:
                new_node = Node(parent=cur, char_to_parent=char)
                cur.son[char] = new_node
                print(f"[Бор] Добавлен узел {new_node.id} из {cur.id}
по символу '{char}'")
            else:
                print(f"[Бор] Уже существовал узел {cur.son[char].id} из
{cur.id} по символу '{char}'")
                cur = cur.son[char]
        cur.is_leaf = True
```

```

        cur.leaf_pattern_numbers.append(pattern_number)
        self.pattern_lengths[pattern_number] = len(s)
        print(f"[Бор] Узел {cur.id} помечен как конец строки
№{pattern_number}")

def get_suff_link(self, v: Node):
    if v.suff_link is None:
        if v == self.root or v.parent == self.root:
            v.suff_link = self.root
        else:
            v.suff_link =
self.get_link(self.get_suff_link(v.parent), v.char_to_parent)
        if v != self.root:
            print(f" [Автомат] Ссылка п: {v.id} ->
{v.suff_link.id}")
        return v.suff_link

def get_link(self, v, c):
    if c not in v.go:
        if c in v.son:
            v.go[c] = v.son[c]
        elif v == self.root:
            v.go[c] = self.root
        else:
            v.go[c] = self.get_link(self.get_suff_link(v), c)
    return v.go[c]

def get_up(self, v):
    if v.up is None:
        sl = self.get_suff_link(v)
        if sl.is_leaf:
            v.up = sl
        elif sl == self.root:
            v.up = self.root
        else:
            v.up = self.get_up(sl)
    if v != self.root:
        print(f" [Автомат] Ссылка up: {v.id} -> {v.up.id}")

```

```

return v.up

def process_text(self, text):
    print(f"{f'\nПОИСК В ТЕКСТЕ {text}':^40}")
    results = []
    cur = self.root
    for i in range(len(text)):
        char = text[i]
        next_node = self.get_link(cur, char)
        print(f"Позиция {i}: '{char}' | Переход {cur.id} ->
{next_node.id}")
        cur = next_node

    temp = cur
    while temp != self.root:
        if temp.is_leaf:
            for p_num in temp.leaf_pattern_numbers:
                start_pos = i - self.pattern_lengths[p_num] + 2
                print(f"Совпадение: Паттерн №{p_num} в
позиции {start_pos}")
                results.append((start_pos, p_num))
            temp = self.get_up(temp)
    return results

def get_max_chains_from_root(self):
    stats = {"max_suff": 0, "max_up": 0}

    def dfs(v):
        if v != self.root:
            curr_suff_len = 0
            temp = v
            while temp != self.root:
                temp = self.get_suff_link(temp)
                curr_suff_len += 1
            stats["max_suff"] = max(stats["max_suff"],
curr_suff_len)

            curr_up_len = 0

```

```

        temp = v
        while temp != self.root:
            temp = self.get_up(temp)
            if temp != self.root:
                curr_up_len += 1
            stats["max_up"] = max(stats["max_up"], curr_up_len)

        for char in sorted(v.son.keys()):
            dfs(v.son[char])

    dfs(self.root)
    return stats["max_suff"], stats["max_up"]

def print_automaton(self):
    print(f"{'ИТОГОВАЯ СТРУКТУРА АВТОМАТА':^60}")

    def show(v, char="ROOT", prefix="", is_last=True):
        sl = self.get_suff_link(v)
        up = self.get_up(v)

        leaf_info = f" (TERM: {v.leaf_pattern_numbers})" if
v.is_leaf else ""
        links = f" | π:{sl.id} up:{up.id}" if v != self.root else
""

        connector = "└─ " if is_last else "├─ "
        print(f"{prefix}{connector}'{char}'
[ID:{v.id}]{leaf_info}{links}")

        new_prefix = prefix + ("  " if is_last else "|  ")
        child_chars = sorted(v.son.keys())
        for i, c in enumerate(child_chars):
            show(v.son[c], c, new_prefix, i == len(child_chars) -
1)

    show(self.root)

if __name__ == "__main__":

```

```

text = input()
n = int(input())
patterns = []
for _ in range(n):
    patterns.append(input())

ac = AhoCorasick()
print(f"{'ПОСТРОЕНИЕ БОРА':^40}")
for i in range(n):
    print(f"Строка '{patterns[i]}'")
    ac.add_string(patterns[i], i+1)

matches = ac.process_text(text)

matches.sort()

for pos, p_idx in matches:
    print(f"{pos} {p_idx}")

suff_max, up_max = ac.get_max_chains_from_root()
print("Максимальная цепочка суффиксных ссылок " + str(suff_max))
print("Максимальная цепочка конечных ссылок " + str(up_max))

ac.print_automaton()

```

Файл ac_wc.py

```

class Node:
    _node_id_counter = 0

    def __init__(self, parent=None, char_to_parent=None):
        self.id = Node._node_id_counter
        Node._node_id_counter += 1
        self.son: dict[str, 'Node'] = {}
        self.go: dict[str, 'Node'] = {}
        self.parent: 'Node' = parent
        self.char_to_parent: str = char_to_parent
        self.suff_link: 'Node' = None
        self.up: 'Node' = None

```

```

        self.is_leaf: bool = False
        self.leaf_pattern_numbers: list[int] = []

class AhoCorasickWildCard:
    def __init__(self):
        Node._node_id_counter = 0
        self.root = Node()
        self.pattern_lengths = {}

    def add_string(self, s, pattern_number):
        cur = self.root
        for char in s:
            if char not in cur.son:
                new_node = Node(parent=cur, char_to_parent=char)
                cur.son[char] = new_node
                print(f"[Бор] Добавлен узел {new_node.id} из {cur.id}
по символу '{char}'")
            else:
                print(f"[Бор] Уже существовал узел {cur.son[char].id} из
{cur.id} по символу '{char}'")
                cur = cur.son[char]
            cur.is_leaf = True
            cur.leaf_pattern_numbers.append(pattern_number)
            self.pattern_lengths[pattern_number] = len(s)
            print(f"[Бор] Узел {cur.id} помечен как конец строки
№{pattern_number}")

    def get_suff_link(self, v: Node):
        if v.suff_link is None:
            if v == self.root or v.parent == self.root:
                v.suff_link = self.root
            else:
                v.suff_link =
self.get_link(self.get_suff_link(v.parent), v.char_to_parent)
            if v != self.root:
                print(f"      [Автомат] Ссылка п: {v.id} ->
{v.suff_link.id}")
        return v.suff_link

```

```

def get_link(self, v, c):
    if c not in v.go:
        if c in v.son:
            v.go[c] = v.son[c]
        elif v == self.root:
            v.go[c] = self.root
        else:
            v.go[c] = self.get_link(self.get_suff_link(v), c)
    return v.go[c]

def get_up(self, v):
    if v.up is None:
        sl = self.get_suff_link(v)
        if sl.is_leaf:
            v.up = sl
        elif sl == self.root:
            v.up = self.root
        else:
            v.up = self.get_up(sl)
    if v != self.root:
        print(f"    [Автомат] Ссылка: {v.id} -> {v.up.id}")
    return v.up

def process_text_with_wildcards(self, text, parts,
full_pattern_len):
    print(f"{f'\nПОИСК В ТЕКСТЕ {text}':^40}")
    counts = [0] * len(text)
    cur = self.root

    for i in range(len(text)):
        char = text[i]
        next_node = self.get_link(cur, char)
        print(f"Позиция {i}: '{char}' | {cur.id} -> {next_node.id}")
        cur = next_node

    temp = cur
    while temp != self.root:

```



```

        if temp.is_leaf:
            for p_idx in temp.leaf_pattern_numbers:
                offset = parts[p_idx][1]
                start_pos = i - self.pattern_lengths[p_idx] -
offset + 1

                print(f"                Совпадение: часть №{p_idx},
возможный старт = {start_pos}")

                if 0 <= start_pos <= len(text) -
full_pattern_len:
                    counts[start_pos] += 1
                    print(f"                Засчитано в
counts[{start_pos}] = {counts[start_pos]}")

                temp = self.get_up(temp)

    return counts

def print_automaton(self):
    print(f"{'\nИТОГОВАЯ СТРУКТУРА АВТОМАТА':^60}")

    def show(v, char="ROOT", prefix="", is_last=True):
        sl = self.get_suff_link(v)
        up = self.get_up(v)

        leaf_info = f"    (TERM:  {v.leaf_pattern_numbers})" if
v.is_leaf else ""
        links = f" | n:{sl.id} up:{up.id}" if v != self.root else
""

        connector = "└─ " if is_last else "├─ "
        print(f"{prefix}{connector}'{char}'
[ID:{v.id}]{leaf_info}{links}")

        new_prefix = prefix + ("    " if is_last else "├─ ")
        child_chars = sorted(v.son.keys())
        for i, c in enumerate(child_chars):

```

```

        show(v.son[c], c, new_prefix, i == len(child_chars) -
1)

    show(self.root)

if __name__ == "__main__":
    text = input()
    pattern = input()
    wildcard = input()

    parts = []
    start = -1

    print(f"{'\nРАЗБИЕНИЕ ШАБЛОНА':^40}")
    for i in range(len(pattern)):
        if pattern[i] != wildcard:
            if start == -1:
                start = i
            else:
                if start != -1:
                    parts.append((pattern[start:i], start))
                    print(f"Часть: '{pattern[start:i]}' с offset {start}")
                    start = -1

                if start != -1:
                    parts.append((pattern[start:], start))
                    print(f"Часть: '{pattern[start:]}' с offset {start}")

    ac = AhoCorasickWildCard()

    print(f"{'\nПОСТРОЕНИЕ БОРА':^40}")
    for i in range(len(parts)):
        print(f"Строка '{parts[i][0]}'")
        ac.add_string(parts[i][0], i)

    counts = ac.process_text_with_wildcards(text, parts, len(pattern))

    print(f"{'\nПРОВЕРКА СОВПАДЕНИЙ':^40}")

```

```
num_parts = len(parts)
for i in range(len(counts)):
    if counts[i] == num_parts:
        print(f"Совпадение полного шаблона с позиции {i + 1}")

ac.print_automaton()
```